

# Final Project: Multi-Track Data Report and Statistical Analysis

---

## Project Overview

You will be provided with an Excel workbook containing student data across multiple academic tracks (e.g., Finance, Data Science, Engineering). Each track is represented as a separate sheet within the workbook, with a consistent structure across sheets. Each row details individual student data, including scores in various subjects (Mathematics, English, Science, History), attendance rates, project scores, income status, cohort affiliation, and more.

Your objective is to develop a robust, reusable Python script that:

- Loads all sheets dynamically from the Excel file
- Cleans and standardizes datasets, handling missing values, waiver tags (e.g., "Waived", "N/A"), and inconsistent data entries
- Merges all individual sheets into a single comprehensive dataset
- Generates a detailed statistical report providing actionable insights across tracks, cohorts, and income categories

The solution must be adaptable to new Excel files from subsequent years without requiring manual code modifications, supporting automated processing.

## Key Objectives

### 1. Data Loading and Cleaning

Import all sheets into a unified pandas DataFrame

Detect and clean common data inconsistencies such as missing or special values (NaN, "", "N/A", "Waived")

Convert and normalize score columns, attendance, and other relevant fields into consistent numeric formats

Add a "Track" column indicating the source sheet for each student's data

### 2. Track-Level Summary Statistics

Compute and present:

- Total number of students per track
- Average subject scores
- Average attendance and project scores
- Pass rates based on the designated "Passed (Y/N)" column

### 3. Cross-Track Comparative Analysis

Visualize distributions of History grades for each track using histograms or boxplots

Compare average Mathematics scores between tracks

Analyze correlations between attendance and project scores per track

### 4. Cohort-Level Analysis

Calculate average grades and pass rates segmented by cohort

Compare academic performance of income-supported students (`IncomeStudent = True`) against others

### 5. Final Report Generation

Export a summary report as a CSV or Excel file encompassing all computed statistics

Optionally generate charts and plots using matplotlib or seaborn libraries for visual interpretation

## Technical Requirements and Tools

Use pandas for data manipulation and integration

Utilize libraries such as openpyxl or xlrd for Excel interactions

Implement data visualizations through matplotlib and seaborn

Apply robust data cleaning methods (e.g., fillna, replace, dtype conversions)

Structure your code modularly with functions or object-oriented programming, enhancing reusability and maintainability

## What to Submit

Your final submission should include:

- Python script file (e.g., `report_generator.py`) capable of executing the entire data pipeline
- Cleaned and merged dataset (CSV format)
- Summary statistics report (CSV or XLSX)
- Visual output files (PNG or embedded charts within Excel files)
- README.md document outlining:
  - - Instructions for running the script
  - - Description of implemented functionalities
  - - Any assumptions or limitations
  - - Details on additional features (if any)
- Optionally, provide the URL of your GitHub repository if you have uploaded your project there

*Ideally, your project should be maintained in a GitHub repository, and the corresponding link shared alongside the submission. Nonetheless, the primary requirement is to submit all project files directly for evaluation.*

### **Bonus Project Enhancements**

Command-line interface menu enabling users to select specific analyses

Dashboard-style Excel output using `xlsxwriter` with enhanced formatting

Automated performance alerts flagging low-performing tracks or cohorts

Trend analysis of performance metrics over multiple terms or academic years, if applicable (to do that, you'll need to generate other years data)

### **Testing and Robustness**

Your script should effortlessly handle future Excel files preserving the same structure but varying in content. It should gracefully skip corrupted or incomplete rows.